📚 Ultimate Guide to File Handling in C (With Deep Explanations & Tricky Exam Questions)

🔹 1. File Operations: Core Functions

1.1 `fopen()` – Opening Files

Purpose: Establishes a connection between your program and a file.

Syntax:

```
FILE *fopen(const char *filename, const char *mode);
```

Modes Explained:

Mode	Description	File Exists?
"r"	Read (text)	Must exist
"w"	Write (text)	Creates/overwrites
"a"	Append (text)	Creates if missing
"rb"	Read (binary)	Must exist
"wb"	Write (binary)	Creates/overwrites

Example:

```
FILE *file = fopen("data.txt", "r");
if (file == NULL) {
    perror("Error"); // Prints detailed error (e.g., "No such file")
    exit(1);
}
```

Key Insight:

- Always check `if (file == NULL)`
- **`perror()`** is better than `printf("Error")`—it explains *why* the file failed to open.

1.2 `fclose()` – Closing Files

Why? Prevents memory leaks and ensures all data is written.

```
fclose(file);
// Failure returns EOF (check with `ferror(file)`)
```

🔹 2. Writing Data to Files

2.1 `fputc()` – Single Character

```
fputc('A', file);  // Writes 'A'
```

2.2 `fputs()` – String (No Auto-Newline)

```
fputs("Hello World!", file);
```

2.3 `fprintf()` – Formatted Output

```
int age = 25;
fprintf(file, "Name: %s, Age: %d", "Alice", age);
```

2.4 `fwrite()` – Binary Data

```
int arr[5] = {1, 2, 3, 4, 5};

fwrite(arr, sizeof(int), 5, file);  // Writes entire array
```

🔹 3. Reading Data from Files

3.1 `fgetc()` – Single Character

```
char ch = fgetc(file);  // Returns EOF on end-of-file
```

3.2 `fgets()` – Line by Line

```
char buffer[100];
while (fgets(buffer, 100, file) != NULL) {  // Correct way
(avoids double reads!)
    printf("%s", buffer);
}
```

3.3 `fscanf()` – Formatted Input

```
int age;
fscanf(file, "Age: %d", &age);  // Reads "Age: 25" → age=25
```

3.4 `fread()` – Binary Data

```
int arr[5];
fread(arr, sizeof(int), 5, file);  // Reads 5 integers
```

🔹 4. File Positioning & Error Handling

4.1 `feof()` – End-of-File Check

Common Mistake:

```
while (!feof(file)) {  // ❌ May read last line twice!
    fgets(buffer, 100, file);
}
```

Correct Approach:

```
while (fgets(buffer, 100, file) != NULL) {  // ✅
    printf("%s", buffer);
}
```

4.2 `ftell()` & `fseek()` – Random Access

```
long pos = ftell(file);  // Current position
fseek(file, 10, SEEK_SET);  // Move to 10th byte from start
```

🔹 5. Binary vs Text Mode

Difference	Text Mode ("r", "w")	Binary Mode ("rb", "wb")
Newlines	Converts \n ↔ \r\n (Windows)	No conversion
Use Case	Text files (.txt)	Images, executables (.jpg, .exe)

Example: Copying an image **must** use binary mode:

```
FILE *src = fopen("pic.jpg", "rb");
FILE *dest = fopen("copy.jpg", "wb");
char ch;
while ((ch = fgetc(src)) != EOF) {
    fputc(ch, dest);
}
```

🔹 6. Advanced: Memory Functions (memcpy, atoi, itoa)

6.1 `memcpy()` – Copy Raw Memory

```
int src[5] = {1, 2, 3, 4, 5};
int dest[5];
memcpy(dest, src, 5 * sizeof(int));  // Copies entire array
```

6.2 `atoi()` vs `itoa()`

Function	Direction	Example
atoi()	String → Int	atoi("123") → 123
itoa()	Int → String	itoa(123, str, 10) → "123"

Note: `itoa()` is non-standard.
Use `sprintf()` instead:

```
char str[20];
sprintf(str, "%d", 123);  // "123"
```

📝 Detailed Answers to Tricky Exam Questions

🔹 Question 1: Why does `while (!feof(file))` print the last line twice?

Problem Code:

```
FILE *file = fopen("data.txt", "r");
while (!feof(file)) {
    char buffer[100];
    fgets(buffer, 100, file);
    printf("%s", buffer);
}
```

Answer:

- **`feof()` only returns true after a read operation fails** (i.e., when you try to read *past* the end of the file).
- **What happens:**
    1. `fgets()` reads the last line successfully → `feof()` is still false.
    2. Loop runs again → `fgets()` tries to read past EOF → fails (buffer unchanged).
    3. `printf()` prints the **old content of buffer again** (double print).

Fix:

```
while (fgets(buffer, 100, file) != NULL) {  // ✅ Stops when fgets fails
    printf("%s", buffer);
}
```

🔹 Question 2: Why does this binary copy fail?

Problem Code:

```
FILE *src = fopen("image.jpg", "r");  // ❌ Text mode
FILE *dest = fopen("copy.jpg", "w"); // ❌ Text mode
```

Answer:

- **Text mode ("r"/"w") corrupts binary files** (e.g., images, .exe) because:
    - It converts \n ↔ \r\n on Windows, altering byte sequences.
    - Binary files **must** use raw bytes ("rb"/"wb").

Fix:

```
FILE *src = fopen("image.jpg", "rb");  // ✅ Binary mode
FILE *dest = fopen("copy.jpg", "wb"); // ✅ Binary mode
```

🔹 Question 3: What does `fseek(file, -10, SEEK_END)` do?

Answer:

- **Moves the file pointer 10 bytes backward from the end** of the file.
- **Use cases:**
    - Reading the last 10 bytes of a file (e.g., file signatures).
    - Appending data at a specific offset before EOF.

Example:

```
fseek(file, -10, SEEK_END);
char last10[10];
fread(last10, 1, 10, file);  // Reads the last 10 bytes
```

🔹 Question 4: Why is perror() better than printf("Error")?

Answer:

- **perror()** prints the **system's error message** (e.g., "No such file", "Permission denied").
- **printf("Error")** gives no context, making debugging harder.

Example:

```
FILE *file = fopen("missing.txt", "r");
if (file == NULL) {
    perror("Failed to open file");  // Output: "Failed to open
file: No such file"
}
```

- **Question 5: How to read a struct from a file?**

Answer:

Use `fread()` to read binary data directly into a struct.

Example:

```
typedef struct {
    char name[50];
    int age;
} Person;

Person p;
FILE *file = fopen("person.dat", "rb");
fread(&p, sizeof(Person), 1, file);  // Reads entire struct
printf("Name: %s, Age: %d", p.name, p.age);
```

🎓 Final Challenge (Hard Exam-Level Questions)

1. **How would you reverse the contents of a text file line-by-line?**
   (Hint: Use fseek() + ftell() to find the file size, then read backwards.)
2. **Why does fscanf(file, "%c", &ch) not skip whitespace? How to fix it?**
   (Hint: Use " %c" with a leading space.)
3. **Write a function that appends a line number to each line in a file.**
   (Example: "1: Hello\n2: World")
4. **Explain the difference between fwrite() and fprintf() for writing integers.**
   (Hint: One writes binary, the other writes text.)
5. **What happens if you open a file in "a+" mode and immediately call fseek(file, 0, SEEK_SET)?**
   (Hint: The position changes, but writes still append!)

🔹 Hard Exam-Level Questions (Answered)

Q1. Reverse a text file line-by-line

Solution:

1. **Read all lines into an array.**
2. **Print the array in reverse.**

```
char lines[100][100];
int count = 0;
while (fgets(lines[count], 100, file) != NULL) {
    count++;
}
for (int i = count - 1; i >= 0; i--) {
    printf("%s", lines[i]);
}
```

Q2. Why `fscanf(file, "%c", &ch)` doesn't skip whitespace?

Answer:

- **%c reads the next character (including spaces/newlines).**
- **Fix:** Add a space before %c to skip whitespace:

```
fscanf(file, " %c", &ch);  // Skips spaces/tabs/newlines
```

Q3. Append line numbers to each line in a file

Solution:

```
FILE *file = fopen("input.txt", "r");
FILE *out = fopen("output.txt", "w");
char line[100];
int line_num = 1;
while (fgets(line, 100, file) != NULL) {
    fprintf(out, "%d: %s", line_num++, line);
}
```

Q4. Difference between `fwrite()` and `fprintf()` for integers

`fwrite()`	`fprintf()`
Writes **binary data** (exact bytes).	Writes **text representation**.
`fwrite(&num, sizeof(int), 1, file);`	`fprintf(file, "%d", num);`
Faster (no conversion).	**Human-readable** (but slower).

Q5. Behavior of `fseek()` in "a+" mode

Answer:

- **"a+" mode allows reading but forces appending for writes.**
- **Even if you `fseek(file, 0, SEEK_SET)`, writes still go to the end!**

```
FILE *file = fopen("data.txt", "a+");
fseek(file, 0, SEEK_SET);  // Move to start
fputs("Hello", file);      // Still appends to end!
```
